

Интеграция frontend и backend

Fetch API, единый API-клиент, JWT и состояние интерфейса

Результаты занятия

- ✓ Понимать контракт между frontend и REST API
- ✓ Отправлять JSON через fetch()
- ✓ Проверять Response.ok и HTTP status
- ✓ Обрабатывать JSON-ответы и ошибки
- ✓ Создавать единый apiFetch()
- ✓ Передавать Bearer access token
- ✓ Связывать login form с Auth API
- ✓ Загружать protected data после login
- ✓ Обрабатывать 401 и 403 на frontend
- ✓ Разделять UI state и server-side access control

Учебные вопросы

- | | | |
|---|------------------------------------|--|
| 1 | Frontend ↔ Backend contract | HTTP method, URL, headers, body, status и JSON |
| 2 | Fetch API | Запросы, Response.ok, status и response.json() |
| 3 | Единый API-клиент | apiFetch(), base URL, JSON и ошибки |
| 4 | JWT в frontend-запросах | Login, accessToken и Authorization: Bearer |
| 5 | UI state и ошибки | Loading, authenticated user, 401, 403 и logout |



Учебный вопрос 1. Frontend ↔ Backend contract

Frontend и backend взаимодействуют через заранее согласованный HTTP-контракт

Что такое API contract

- API contract — согласованный формат запросов и ответов между frontend и backend.
- Frontend должен знать HTTP method и URL.
- Frontend должен знать формат request body.
- Backend определяет HTTP status и JSON response.
- Обе стороны должны одинаково трактовать ошибки.
- Изменение API-контракта требует согласованного изменения frontend.

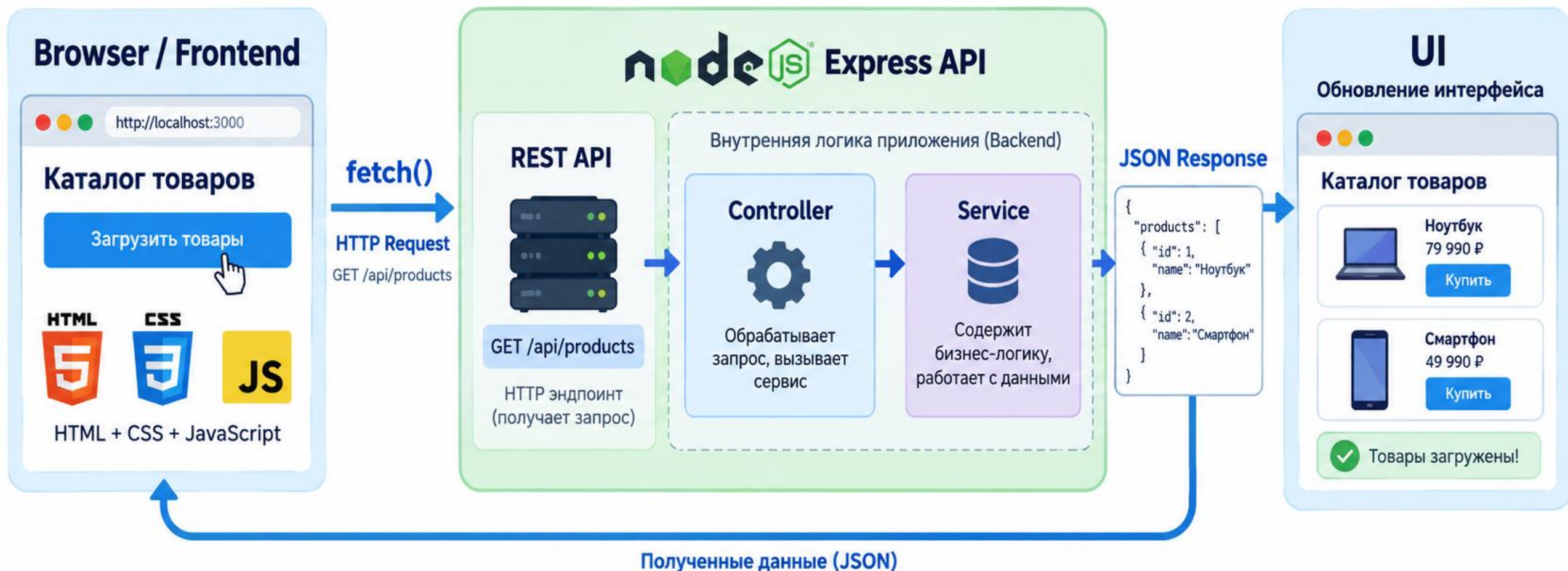
Контракт Internet Shop API

Действие	Метод	URL	Результат
Login	POST	/api/auth/login	user + accessToken
Текущий User	GET	/api/auth/me	user
Каталог	GET	/api/products	Product[]
Создать заказ	POST	/api/orders	Order
Мои заказы	GET	/api/orders/my	Order[]

Request и Response

- Request идёт от frontend к backend.
- Request содержит method, URL, headers и при необходимости body.
- Response идёт от backend к frontend.
- Response содержит status, headers и body.
- Для нашего приложения body обычно передаётся как JSON.

Frontend → HTTP Request → Express API → JSON Response → UI



Frontend и Backend: разделение ответственности

Frontend

- Форма и пользовательский ввод
- `fetch()` запросы
- Отображение `loading/error/data`
- Навигация и состояние UI
- Отправка `access token`

Backend

- Validation
- Authentication и Authorization
- Бизнес-логика
- Prisma и PostgreSQL
- HTTP status и JSON

Frontend не должен повторять бизнес-логику backend

- Frontend может предварительно проверить форму для удобства.
- Но backend всё равно повторно валидирует входные данные.
- Frontend может скрыть admin-кнопку.
- Но backend всё равно проверяет `requireRole`.
- Источник истины для безопасности и данных находится на сервере.



Учебный вопрос 2. Fetch API

fetch() связывает browser JavaScript с REST API

Простейший GET

JAVASCRIPT

```
const response = await fetch("http://localhost:3000/api/products");  
const products = await response.json();  
  
console.log(products);
```

Что важно

- `fetch()` возвращает Promise с объектом Response.
- `response.json()` асинхронно читает JSON body.
- GET используется по умолчанию.

fetch() и HTTP-ошибки

- `fetch()` обычно не отклоняет Promise только потому, что сервер вернул 404 или 500.
- Нужно явно проверить `response.ok` или `response.status`.
- `response.ok` равен `true` для HTTP status 200–299.
- Сетевая ошибка и HTTP error — разные ситуации.

Проверяем response.ok

JAVASCRIPT

```
const response = await fetch(url);

if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}

const data = await response.json();
```

Что важно

- HTTP 404 всё ещё создаёт объект Response.
- response.ok позволяет отделить успешный HTTP-ответ.
- После успешной проверки читаем JSON.

fetch(): network error vs HTTP error vs successful JSON

Один и тот же код `fetch()` может привести к разным результатам. Важно проверять `response.ok` и обрабатывать ошибки.



Internet Shop

Learn. Build. Deploy.

Frontend (Browser)



`fetch('/api/orders')`

HTTP request

```
try {
  const response = await fetch('/api/orders', { headers: { 'Authorization': 'Bearer ' + token } });
  // обработка в зависимости от результата
} catch (error) {
  // срабатывает только при сетевой ошибке
}
```

Запрос к API



Express API
`/api/orders`



1. Network Error

Сетевая ошибка (нет соединения)



Интернет



API недоступен

- Сервер не запущен
- Нет интернета
- Неверный домен
- Таймаут подключения

```
try {
  await fetch('http://localhost:3000/api/orders');
} catch (error) {
  console.error('Network error:', error);
}
```

Результат

`fetch()` выбрасывает исключение (exception)

- `TypeError: Failed to fetch`
- `TypeError: NetworkError when attempting to fetch resource`
- и другие ошибки сети



2. HTTP Error (401, 404, 500)

Сервер ответил с кодом ошибки



HTTP request



Express API

Примеры:

- 401 Unauthorized (нет токена / неверный токен)
- 404 Not Found (ресурс не найден)
- 500 Internal Server Error (ошибка сервера)

```
const response = await fetch('/api/orders');
console.log(response.ok); // false
console.log(response.status); // 401, 404, 500
const text = await response.text();
```

Результат

`fetch()` не выбрасывает исключение.

Возвращается объект `Response`:

- `response.ok = false`
- `response.status = 401` (или 404, 500 и т.д.)
- можно прочитать текст ошибки: `await response.text()`

```
Response {
  ok: false,
  status: 401,
  statusText: "Unauthorized",
  ...
}
```



3. Successful JSON (200)

Успешный ответ с JSON



HTTP request

JSON response



Express API

```
const response = await fetch('/api/orders');
console.log(response.ok); // true
console.log(response.status); // 200
const data = await response.json();
console.log(data);
```

Результат

`fetch()` возвращает успешный `Response`:

- `response.ok = true`
- `response.status = 200`
- можно вызвать `await response.json()` и получить данные

```
Response {
  ok: true,
  status: 200,
  statusText: "OK",
  ...
}
```



```
[
  { "id": 1, "name": "Ноутбук", "price": 499 },
  { "id": 2, "name": "Мышь", "price": 25 }
]
```



Проверяйте `response.ok` и обрабатывайте ошибки в обоих случаях: сетевые (catch) и HTTP (`ok = false`).

```
if (!response.ok) {
  throw new Error('HTTP error: ${response.status}');
}
```



Надёжное приложение
корректно обрабатывает все варианты

POST JSON

JAVASCRIPT

```
const response = await fetch("http://localhost:3000/api/auth/login", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify({ email, password })  
});
```

Что важно

- method явно задаёт POST.
- Content-Type сообщает backend формат body.
- JSON.stringify() превращает JavaScript object в JSON string.

Не каждый response обязан иметь JSON

- API Internet Shop в большинстве случаев возвращает JSON.
- Но код frontend не должен слепо считать любой ответ JSON.
- 204 No Content не содержит body.
- При проектировании API полезно использовать единый формат ошибок.



Учебный вопрос 3. Единый API-клиент

Не повторяем `fetch()` и обработку ошибок во всех компонентах

Проблема дублирования

- Без общего API-клиента каждый файл повторяет base URL.
- Повторяются JSON headers.
- Повторяется Authorization header.
- Повторяется проверка response.ok.
- Повторяется разбор ошибок.
- Изменение API становится сложнее.

config.js

JAVASCRIPT

```
export const API_URL = "http://localhost:3000/api";
```

apiFetch(): базовая версия

JAVASCRIPT

```
import { API_URL } from "./config.js";

export async function apiFetch(path, options = {}) {
  const response = await fetch(`${API_URL}${path}`, options);

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.error ?? `HTTP ${response.status}`);
  }

  return data;
}
```

Что важно

- path добавляется к общему API_URL.
- Функция централизует проверку response.ok.
- Внешний код получает уже разобранные данные или exception.

Добавляем JSON автоматически

- Для POST/PATCH удобно передавать обычный JavaScript object.
- `apiFetch` может самостоятельно выполнить `JSON.stringify()`.
- `apiFetch` может автоматически установить `Content-Type`.
- Компонентам интерфейса не нужно знать технические детали каждого запроса.

apiFetch() с JSON body

```
export async function apiFetch(path, { body, headers = {}, ...options } = {}) {
  const response = await fetch(`${API_URL}${path}`, {
    ...options,
    headers: {
      ...(body ? { "Content-Type": "application/json" } : {}),
      ...headers
    },
    body: body ? JSON.stringify(body) : undefined
  });

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.error ?? `HTTP ${response.status}`);
  }

  return data;
}
```

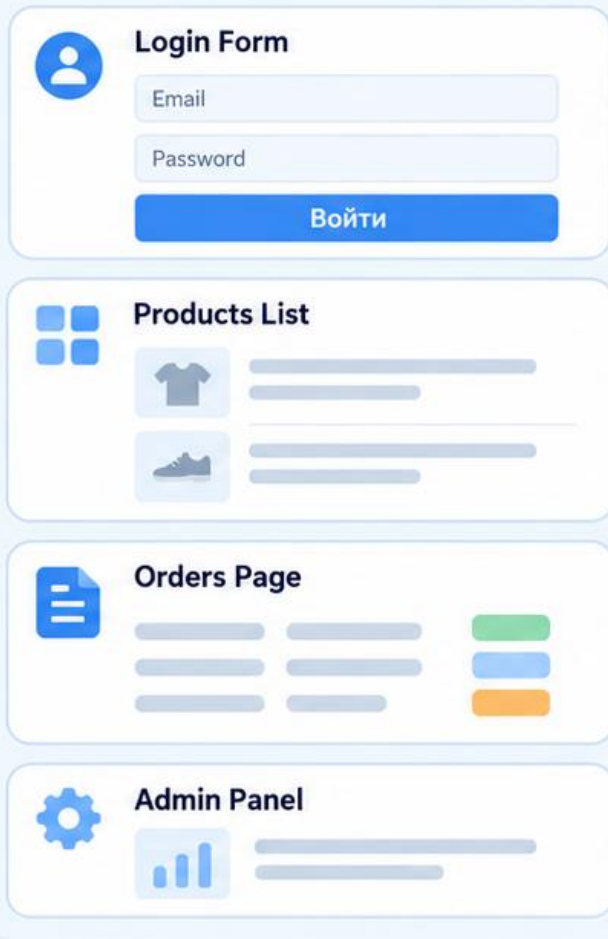
Что важно

- body передаётся как object.
- Функция сама выполняет JSON.stringify().
- Дополнительные headers можно передать извне.

UI components → **apiFetch()** → REST API

UI components (Frontend)

Разные компоненты используют один `apiFetch()`



запрос через
`apiFetch()`

запрос через
`apiFetch()`

запрос через
`apiFetch()`

запрос через
`apiFetch()`

apiFetch()

Общий модуль для работы с API



API_URL

базовый адрес
API



JSON

автоматическая
сериализация
и парсинг



headers

общие заголовки
(например, Auth)



ошибки

централизованная
обработка ошибок

HTTP запрос
(fetch)

JSON ответ
(данные)

REST API (Express API)

Сервер обрабатывает запросы



GET /api/products
POST /api/login
GET /api/orders
...



единая точка доступа к API

Все компоненты обращаются к API через один общий модуль `apiFetch()`.



компоненты не дублируют `fetch()`

Не нужно писать `fetch()` в каждом компоненте — вся логика в одном месте.



общая обработка ошибок и заголовков

Единые заголовки (headers) и обработка ошибок для всех запросов.



Учебный вопрос 4. JWT в frontend-запросах

После login protected requests должны передавать access token

Полный frontend login flow

- 1 Получить форму** Пользователь вводит email и password.
- 2 Отправить login** Frontend вызывает POST /api/auth/login.
- 3 Получить ответ** Backend возвращает user и accessToken.
- 4 Обновить state** Frontend сохраняет User и token в текущем runtime state.
- 5 Выполнять protected requests** apiFetch добавляет Authorization: Bearer.

authState.js: состояние текущей страницы

JAVASCRIPT

```
let accessToken = null;
let currentUser = null;

export function setAuth(data) {
  accessToken = data.accessToken;
  currentUser = data.user;
}

export function clearAuth() {
  accessToken = null;
  currentUser = null;
}

export function getAccessToken() {
  return accessToken;
}

export function getCurrentUser() {
  return currentUser;
}
```

Что важно

- Для учебной схемы token хранится только в памяти JavaScript.
- После reload страницы состояние исчезает.
- Это позволяет сосредоточиться на интеграции без небезопасного persistent token storage.

Почему не сохраняем JWT в localStorage в этой схеме

- localStorage доступен JavaScript-коду origin.
- При XSS злоумышленник может прочитать сохранённый token.
- OWASP рекомендует не хранить authentication tokens и session identifiers в localStorage/sessionStorage.
- Для production browser-сценариев часто предпочитают HttpOnly Secure cookies или BFF.
- В этой лекции accessToken живёт только в памяти страницы.

apiFetch добавляет Bearer token

JAVASCRIPT

```
import { getAccessToken } from "../authState.js";

export async function apiFetch(path, { body, headers = {}, ...options } = {}) {
  const token = getAccessToken();

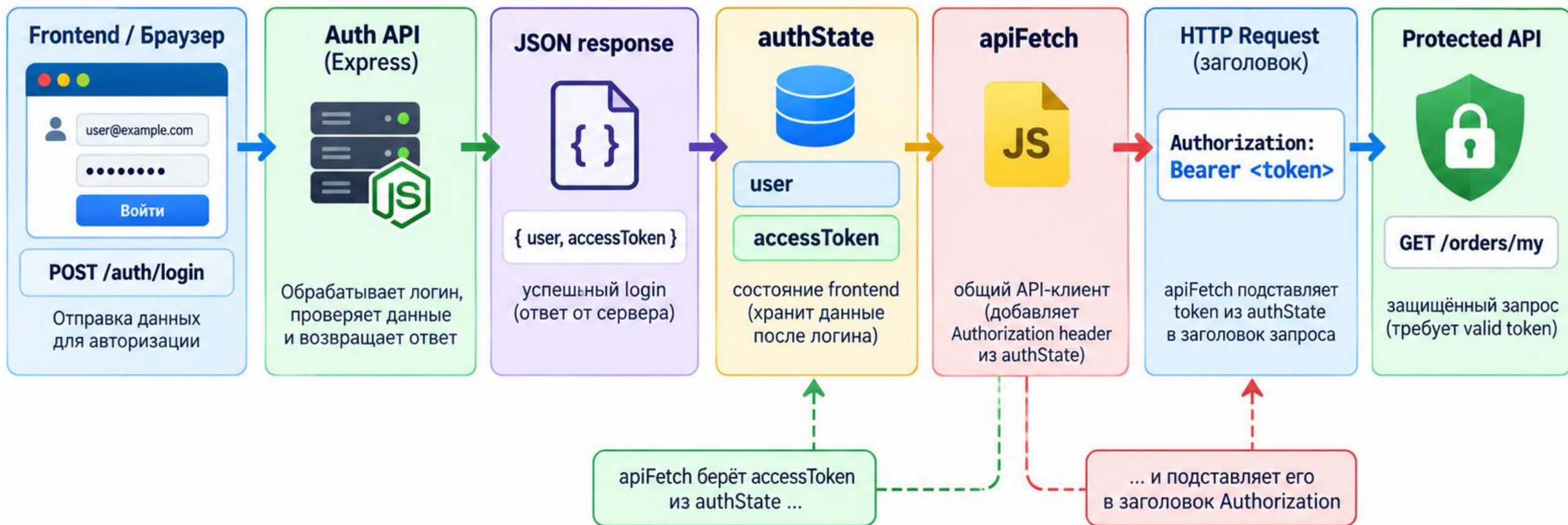
  const response = await fetch(`${API_URL}${path}`, {
    ...options,
    headers: {
      ...(body ? { "Content-Type": "application/json" } : {}),
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...headers
    },
    body: body ? JSON.stringify(body) : undefined
  });

  // обработка response дальше
}
```

Что важно

- Публичный запрос работает и без token.
- При наличии token автоматически формируется Authorization header.
- Компоненты не повторяют Bearer-логику.

Login response → authState → Authorization header → protected API



В результате: после успешного логина frontend сохраняет user и accessToken, а apiFetch автоматически добавляет Bearer token в каждый защищённый запрос (например, /orders/my).

*Один раз вошёл —
и API работает!*

login() на frontend

JAVASCRIPT

```
export async function login(email, password) {  
  const data = await apiFetch("/auth/login", {  
    method: "POST",  
    body: { email, password }  
  });  
  
  setAuth(data);  
  return data.user;  
}
```

Что важно

- login() использует общий apiFetch().
- При успехе сохраняются user и accessToken.
- UI получает безопасный User object.

Загрузка protected resource

JAVASCRIPT

```
export async function loadMyOrders() {  
  return apiFetch("/orders/my");  
}
```

Что важно

- Функция выглядит как обычный GET.
- Authorization header добавляется централизованно.
- Backend сам проверяет requireAuth и ownership.

Protected request

Запрос: GET /api/orders/my

Код • HTTP

```
GET /api/orders/my
Authorization: Bearer <accessToken>
```

Результат • JSON

```
[
  {
    "id": 15,
    "status": "new",
    "items": []
  }
]
```

Пояснение

- Frontend не передаёт userId: backend использует identity из JWT.



Учебный вопрос 5. UI state и ошибки

Интерфейс должен отражать состояние запроса, но не подменять безопасность backend

Основные UI-состояния

 **idle**

Запрос ещё не начат.

 **loading**

Ожидаем ответ backend.

 **success**

Данные получены и отображаются.

 **error**

API вернул ошибку или произошла сетевая проблема.

Login form: обработчик submit

JAVASCRIPT

```
form.addEventListener("submit", async (event) => {
  event.preventDefault();
  errorBox.textContent = "";
  loginButton.disabled = true;

  try {
    const user = await login(emailInput.value, passwordInput.value);
    showAuthenticatedUI(user);
  } catch (error) {
    errorBox.textContent = error.message;
  } finally {
    loginButton.disabled = false;
  }
});
```

Что важно

- `preventDefault()` отменяет обычную отправку HTML form.
- Кнопка временно отключается во время запроса.
- Ошибка отображается пользователю.
- `finally` выполняется и при `success`, и при `error`.

401 и 403 должны менять поведение UI по-разному

- 401 означает, что Authentication отсутствует или больше недействительна.
- Frontend может очистить auth state и показать login form.
- 403 означает, что User известен, но действие запрещено.
- При 403 не нужно автоматически считать пользователя logged out.
- Frontend показывает сообщение о недостаточных правах.

APIError co status

JAVASCRIPT

```
export class APIError extends Error {  
  constructor(message, status) {  
    super(message);  
    this.status = status;  
  }  
}  
  
// внутри apiFetch()  
if (!response.ok) {  
  throw new APIError(  
    data.error ?? `HTTP ${response.status}`,  
    response.status  
  );  
}
```

Что важно

- Frontend получает не только message, но и HTTP status.
- Это позволяет различать 401, 403, 404 и другие ответы.

Централизованная обработка 401

JAVASCRIPT

```
if (response.status === 401) {  
  clearAuth();  
}  
  
if (!response.ok) {  
  throw new APIError(  
    data.error ?? `HTTP ${response.status}`,  
    response.status  
  );  
}
```

Что важно

- Expired или invalid JWT очищает локальное auth state.
- Далее UI может показать форму login.
- 403 не очищает Authentication автоматически.

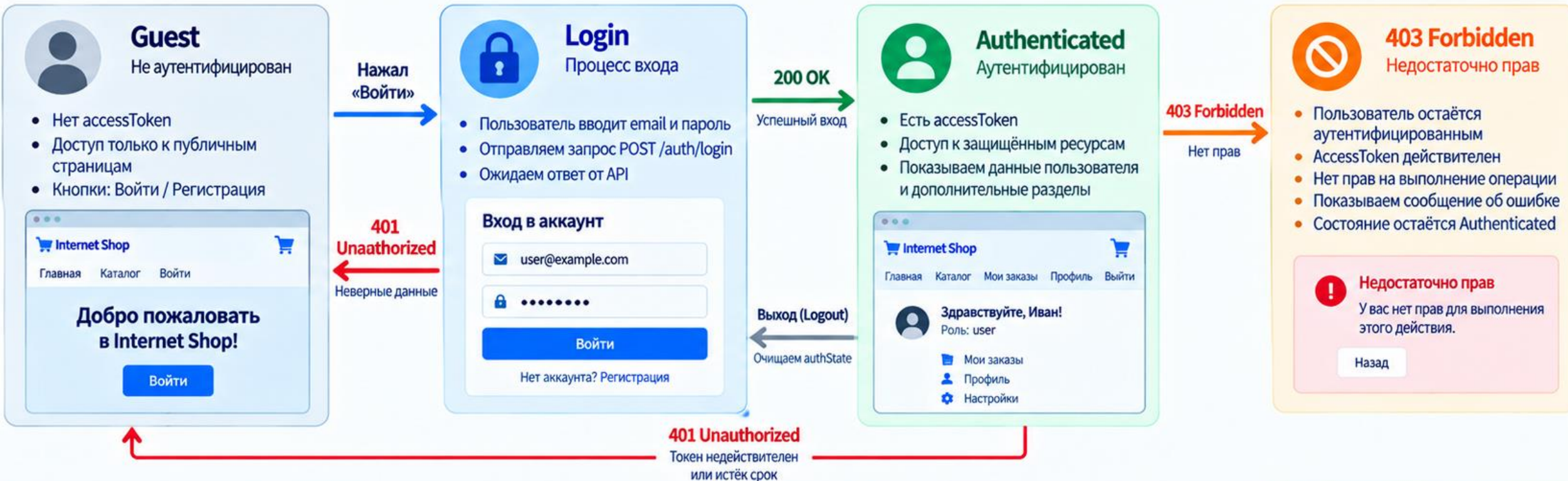
Frontend state machine: аутентификация и авторизация

Состояния, переходы и реакция на ответы API (200 / 401 / 403).



Internet Shop

Secure. Simple. For Everyone.



Примеры API запросов

```
// Login
POST /auth/login
{
  "email": "user@example.com",
  "password": "*****"
}
```

```
// Получение своих заказов (защищённый ресурс)
GET /orders/my
Authorization: Bearer <accessToken>
```

Переходы между состояниями

- Guest → Login** (пользователь нажал «Войти»)
- Login → Authenticated** (200 OK — успешный вход)
- Login → Guest** (401 — неверные данные)
- Authenticated → Authenticated** (403 — нет прав, показываем сообщение, пользователь остаётся в системе)
- Authenticated → Guest** (401 — токен недействителен или истёк срок / пользователь вышел)

Коды ответов и поведение

- 200 OK**
 - Успешный вход
 - Переход в состояние Authenticated
- 401 Unauthorized**
 - Неверные учетные данные или недействительный токен
 - Переход в состояние Guest
- 403 Forbidden**
 - Пользователь аутентифицирован, но нет прав
 - Остаёмся в состоянии Authenticated
 - Показываем сообщение
- Другие (404, 500)**
 - Показываем сообщение об ошибке
 - Состояние не меняется

Хранение состояния (пример)

```
// authState (в памяти)
let authState = {
  user: null,
  accessToken: null
};

function setAuth(user, token) {
  authState.user = user;
  authState.accessToken = token;
}

function logout() {
  authState.user = null;
  authState.accessToken = null;
}

! Токен хранится в памяти (не в localStorage) для большей безопасности.
```

Logout в текущей учебной схеме

- Access token хранится только в памяти страницы.
- Logout вызывает `clearAuth()`.
- UI возвращается в `guest state`.
- Сам JWT технически может оставаться валидным до `exp`, если где-то существует его копия.
- Полный `server-side lifecycle logout/refresh` будет отдельной архитектурной задачей.

logout()

JAVASCRIPT

```
export function logout() {  
  clearAuth();  
  showGuestUI();  
}
```

Что важно

- Очищается currentUser.
- Очищается accessToken.
- Protected UI скрывается.

Что происходит после reload страницы

- In-memory token теряется.
- Пользователь снова становится guest для frontend.
- Это ожидаемое ограничение текущей учебной реализации.
- Persisted login требует другой архитектуры хранения credentials.
- Для production следует отдельно проектировать cookies, refresh tokens или BFF.

Мини-практика: связать UI с Internet Shop API

Создайте страницу login и страницу моих заказов, использующие общий apiFetch().

Задание

- Создать config.js
- Создать api.js с apiFetch()
- Создать authState.js
- Подключить login form
- Сохранить user и accessToken в памяти
- Загрузить /api/orders/my
- Показать loading/error/data
- Обработать 401 и logout

Критерии проверки

- Login выполняется через fetch()
- Bearer token добавляется автоматически
- Protected orders загружаются после login
- 401 очищает auth state
- 403 показывается как недостаток прав
- JWT не сохраняется в localStorage

Типичные ошибки интеграции

Ошибка	Симптом	Исправление
Не проверяется response.ok	404/401 воспринимается как success	Проверять status/ok
Нет Content-Type	req.body пустой или неверный	application/json
Bearer добавляется вручную везде	Дублирование	Централизовать apiFetch
JWT хранится в localStorage	Риск кражи при XSS	Для учебного варианта хранить в памяти
Frontend доверяет своей role	Можно обойти UI	Проверять права на backend

Полный frontend ↔ backend flow Internet Shop

От авторизации до отображения данных. Основной поток и обработка ошибок (401 / 403).



Internet Shop

Secure. Simple. For Everyone.



Frontend (Browser)

HTML / CSS / JavaScript (fetch)



1 Login form

Пользователь вводит email и пароль



Internet Shop

user@example.com

Войти

Нет аккаунта? Регистрация



10 Отображение данных в UI

Обновляем страницу: список заказов

Мои заказы

№	Дата	Статус	Сумма	
101	12.09.2026	pending	129.00	Детали
102	10.09.2026	paid	59.50	Детали



2 apiFetch()

Единая функция для запросов к API

```
// api.js
async function apiFetch(
  url, options = {}
) {
  const token = authState
    .accessToken;
  const headers = {
    'Content-Type': 'application/json',
    ...(token ? { 'Authorization':
      `Bearer ${token}` } : {}),
    ...options.headers
  };
  const res = await fetch(API_URL + url,
    { ...options, headers });
  if (!res.ok) throw await handleError(res);
  return res.json();
}
```



3 Auth API

POST /auth/login

```
POST /auth/login
Content-Type: application/json
{
  "email": "user@example.com",
  "password": "*****"
}
```

Response 200 OK

```
{
  "user": { "id": 1,
    "name": "Иван", "role": "user"
  },
  "accessToken": "eyJhbGciOi..."
}
```

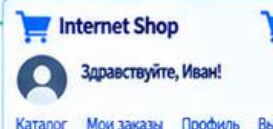


4 JWT → frontend state

Сохраняем данные в состоянии приложения

```
authState (в памяти)
{
  user: { id: 1, name: 'Иван',
    role: 'user' },
  accessToken: "eyJhbGciOi..."
}
```

- ✓ Пользователь аутентифицирован
- ✓ Токен хранится в памяти (не в localStorage)
- ✓ Интерфейс обновляется (показываем имя, меню, кнопки)



Далее делаем запрос к защищённому ресурсу

Backend (Express.js)



Express



5 Protected request

GET /orders/my

```
GET /orders/my
Authorization: Bearer
eyJhbGciOi...
Accept: application/json
```



6 requireAuth

Middleware

```
function requireAuth(
  req, res, next
) {
  try {
    const token =
      getToken(req);
    req.user = jwt.verify(
      token, process.env
        .JWT_SECRET);
    next();
  } catch (e) {
    return res.status(401)
      .json({ error:
        'Unauthorized' });
  }
}
```

Response 200 OK

```
[
  { "id": 101, "status": "pending", "total": 129.00, "items": [ ... ] },
  { "id": 102, "status": "paid", "total": 59.50, "items": [ ... ] }
]
```



7 Controller

Обработка запроса

```
async function
getMyOrders(req, res) {
  const userId = req.user.id;
  const orders = await
    orderService.getOrdersByUserId(
      userId);
  res.json(orders);
}
```



8 Service

Бизнес-логика

```
async function
getOrdersByUserId(
  userId
) {
  return prisma.order
    .findMany({
      where: { userId },
      include: { items: true }
    });
}
```



9 Prisma / PostgreSQL

Доступ к данным



PostgreSQL (через Prisma)

Данные из БД через Prisma



Ошибка 401 — Unauthorized

Токен отсутствует, недействителен или истёк

Response 401

```
{
  "error": "Unauthorized",
  "message": "Invalid or expired token"
}
```

Перенаправляем на страницу входа

Войдите в аккаунт

Войти

Когда возникает:

- Нет токена в запросе
- Токен недействителен
- Токен истёк
- Неверная подпись JWT



Ошибка 403 — Forbidden

Пользователь аутентифицирован, но нет прав

Response 403

```
{
  "error": "Forbidden",
  "message": "Недостаточно прав"
}
```

Сообщаем об ошибке (пользователь остаётся авторизован)



Недостаточно прав

У вас нет прав для выполнения этого действия.

Назад

Когда возникает:

- Токен действителен
- Пользователь аутентифицирован
- Нет прав на ресурс/операцию (например, доступ только для admin)
- Состояние пользователя не меняется (остаётся authenticated)



Итог

Единый, безопасный и понятный flow взаимодействия frontend и backend.



1 Вход



2 apiFetch()



3 Auth API



4 authState



5 Защищённый запрос



6 requireAuth



7 Controller



8 Service



9 Prisma / БД



10 Данные в UI

Контрольные вопросы

1

1. Что входит в API contract?

2

2. Что возвращает `fetch()`?

3

3. Почему HTTP 404 не обязательно приводит к `reject` `fetch Promise`?

4

4. Для чего используется `response.ok`?

5

5. Зачем нужен единый `apiFetch()`?

6

6. Как frontend передаёт JWT backend?

7

7. Почему `userId` не нужно отправлять при `GET /orders/my`?

8

8. Чем обработка 401 отличается от 403?

9

9. Почему `hidden admin button` не является механизмом безопасности?

10

10. Почему в учебной схеме JWT не сохраняется в `localStorage`?

Итоги занятия

- 1 Frontend и backend взаимодействуют через HTTP API contract.
- 2 Fetch API возвращает Response и требует явной проверки HTTP status.
- 3 response.ok удобно использовать для определения успешного ответа.
- 4 apiFetch() централизует URL, JSON, Bearer token и обработку ошибок.
- 5 После login frontend получает User и accessToken.
- 6 Protected requests передают Authorization: Bearer <token>.
- 7 Backend остаётся единственным доверенным механизмом Authentication и Authorization.
- 8 401 обычно переводит frontend обратно в guest state.
- 9 403 означает недостаточные права, но не потерю Authentication.
- 10 Для учебной схемы access token хранится только в памяти страницы.